

SemaGraph v0.6

Kernel deterministico per ancoraggio di stati osservati e compressione di catene di transizione in Q_6

Bozza tecnica per sviluppo e validazione

Giugno 2026

Sommario

SemaGraph v0.6 propone un kernel deterministico per rappresentare cambiamenti osservati come catene finite di transizioni in uno spazio booleano a sei dimensioni, indicato come Q_6 . Ogni stato è codificato da una parola binaria di sei bit; ogni transizione diretta tra due stati è rappresentata da una maschera XOR; ogni catena di trasformazione è comprimibile in una firma operativa costituita da stato sorgente, stato target, maschere ordinate, mutazione netta e distanza cumulativa. Il sistema non usa modelli linguistici per inferire stati o parametri: l'ancoraggio iniziale deriva da osservazioni fisiche, sensori, soglie o parametrizzazioni dichiarate. I modelli linguistici possono intervenire soltanto a valle, per sintetizzare policy o spiegazioni su firme già calcolate. L'obiettivo è ridurre inferenza ricorrente, drift interpretativo e costo computazionale nella lettura di traiettorie dinamiche, mantenendo tracciabilità verso i dati osservati.

1 Premessa

Molti sistemi complessi producono sequenze di stati più che singoli stati isolati. Processi organizzativi, modelli finanziari, fenomeni biologici, eventi fisici discretizzati e simulazioni stocastiche possono essere osservati come traiettorie: una configurazione iniziale cambia, attraversa regimi intermedi e raggiunge una configurazione finale. Il problema non è soltanto classificare lo stato corrente, ma rappresentare come il sistema vi è arrivato.

SemaGraph v0.6 nasce per formalizzare questo passaggio. Il kernel non pretende di sostituire modelli fisici, chimici, biologici, statistici o linguistici. Esso opera come layer di compressione transizionale: prende stati già osservati o parametrizzati, calcola transizioni deterministiche, comprime catene e genera firme confrontabili.

La tesi principale è la seguente: se gli stati sono ancorati deterministicamente a osservazioni o parametri noti, allora la catena di trasformazione può essere rappresentata come sequenza di transizioni finite, replayable e comprimibili. L'inferenza viene confinata fuori dal nucleo transizionale.

2 Modello formale

2.1 Spazio degli stati

Sia $B = \{0, 1\}$. Lo spazio State64 è definito come:

$$Q_6 = B^6. \quad (1)$$

Ogni stato $s \in Q_6$ è una tupla ordinata di sei bit:

$$s = (b_1, b_2, b_3, b_4, b_5, b_6), \quad b_i \in \{0, 1\}. \quad (2)$$

Lo spazio contiene esattamente:

$$|Q_6| = 2^6 = 64 \quad (3)$$

stati possibili. In implementazione, ogni stato può essere rappresentato come intero unsigned a 8 bit usando soltanto i sei bit meno significativi o come stringa canonica di sei caratteri binari.

2.2 Transizione elementare

Date due configurazioni $s, t \in Q_6$, la transizione diretta da s a t è definita dalla maschera:

$$m = s \oplus t, \quad (4)$$

dove $\oplus$ indica XOR bit-a-bit. La maschera m specifica quali posizioni cambiano. Se $m_i = 1$, il bit i è mutato; se $m_i = 0$, il bit i è conservato.

La distanza di trasformazione è la distanza di Hamming:

$$d(s, t) = \sum_{i=1}^6 |s_i - t_i| = \text{popcount}(s \oplus t). \quad (5)$$

Questa misura non interpreta il cambiamento: misura quante coordinate dello stato sono mutate.

2.3 Matrice completa delle transizioni

Poiché Q_6 contiene 64 stati, il numero di coppie ordinate stato-stato è:

$$64 \times 64 = 4096. \quad (6)$$

Queste 4096 coppie costituiscono la base completa delle transizioni dirette, inclusa l'identità $s \rightarrow s$. Le transizioni trasformatrici non identitarie sono:

$$64 \times 63 = 4032. \quad (7)$$

L'indice computazionale di una transizione può essere definito come:

$$I(s, t) = 64 \cdot \text{idx}(s) + \text{idx}(t), \quad (8)$$

dove $\text{idx}(s) \in [0, 63]$ è la rappresentazione intera dello stato sorgente.

2.4 Modularità 3+3

Ogni stato può essere scomposto in due moduli da tre bit:

$$s = (l, u), \quad l \in B^3, \quad u \in B^3. \quad (9)$$

La scomposizione 3+3 non introduce significato ereditato; è una proprietà combinatoria utile per classificare se una transizione rimane interna a un modulo o attraversa entrambi i moduli. Date le distanze:

$$d_l = d(l_s, l_t), \quad d_u = d(u_s, u_t), \quad (10)$$

possiamo classificare il campo di mutazione come: nessun cambiamento, solo modulo inferiore, solo modulo superiore o entrambi i moduli.

3 Ancoraggio deterministico degli stati

3.1 Osservazione e parametrizzazione

SemaGraph v0.6 assume che lo stato non venga inferito linguisticamente. Lo stato è prodotto da regole di ancoraggio esplicite applicate a parametri osservati:

$$A : X \rightarrow Q_6, \quad (11)$$

dove X è lo spazio dei vettori osservativi o parametrici. Ogni bit b_i è calcolato da una regola dichiarata:

$$b_i = r_i(x), \quad r_i : X \rightarrow \{0, 1\}. \quad (12)$$

Una regola può essere una soglia, una condizione booleana, una categoria nota o un predicato su un parametro.

3.2 Replayability

Dato lo stesso vettore osservato x e la stessa versione delle regole $R = (r_1, \dots, r_6)$, il sistema produce sempre lo stesso stato:

$$A_R(x) = s. \quad (13)$$

La determinazione dello stato è quindi replayable. L'incertezza di misura non viene negata: deve essere registrata come tolleranza, unità, fonte, timestamp e versione della regola. Tuttavia, questa incertezza appartiene al livello di misura, non al kernel di transizione.

3.3 Boundary del modello linguistico

Il modello linguistico non assegna stati, non modifica parametri e non inferisce misure mancanti. Può intervenire a valle:

- per spiegare una firma transizionale;
- per proporre policy operative;
- per sintetizzare implicazioni leggibili;
- per confrontare policy già approvate.

Questa separazione impedisce che l'inferenza linguistica si propaghi dentro la catena deterministica.

4 Compressione delle catene di transizione

4.1 Catena ordinata

Una traiettoria osservata è una sequenza ordinata:

$$C = (s_0, s_1, \dots, s_n), \quad s_i \in Q_6. \quad (14)$$

Ogni passaggio produce una maschera:

$$m_i = s_{i-1} \oplus s_i. \quad (15)$$

La catena può quindi essere rappresentata come:

$$C_M = (m_1, m_2, \dots, m_n). \quad (16)$$

4.2 Mutazione netta e distanza cumulativa

La mutazione netta è:

$$M_{net} = m_1 \oplus m_2 \oplus \dots \oplus m_n = s_0 \oplus s_n. \quad (17)$$

La distanza cumulativa è:

$$D_C = \sum_{i=1}^n \text{popcount}(m_i). \quad (18)$$

La mutazione netta esprime l'effetto complessivo; la sequenza ordinata conserva il percorso. Questi due livelli devono rimanere separati, perché catene diverse possono avere la stessa mutazione netta.

4.3 Firma operativa

Una firma transizionale minima può essere definita come:

$$\Sigma(C) = (s_0, s_n, (m_1, \dots, m_n), M_{net}, D_C). \quad (19)$$

Questa firma non sostituisce il dato grezzo, ma lo indicizza. Il dato originale resta necessario per audit, verifica e revisione.

5 Architettura computazionale

5.1 Livelli

L'architettura v0.6 distingue quattro livelli:

1. osservazione e misura;
2. ancoraggio deterministico a State64;
3. processore di transizioni e compressione;
4. sintesi di policy a valle.

Il terzo livello è il nucleo computazionale. Esso può essere implementato con operazioni CPU elementari: XOR, AND, SHIFT, popcount e lookup table.

5.2 Core Rust

La versione v0.6 introduce un core Rust di fattibilità. La rappresentazione minima è:

```
state: u8 // valori ammessi 0..63
mutation_mask: u8 // valori ammessi 0..63
index: u16 // valori ammessi 0..4095
```

Le operazioni base sono:

```
next_state = state ^ mutation_mask
mutation = source ^ target
distance = popcount(source ^ target)
index = source * 64 + target
```

Rust è adatto perché permette controllo numerico, portabilità, test di parità, compilazione verso WebAssembly e interfacce C ABI senza introdurre gestione manuale fragile della memoria.

5.3 TypeScript come reference layer

TypeScript rimane il livello di riferimento per specifica, API, esempi, JSON schema, integrazione web e strumenti AI. Rust non sostituisce immediatamente TypeScript: viene introdotto come candidato a diventare autorità computazionale dopo test di parità.

6 Tool AI e sintesi di policy

La versione v0.6 include un tool adapter per GPT/Claude. Il suo principio operativo è semplice:

Il modello può usare il kernel per processare stati e catene, ma non può alterare stati osservati, maschere o firme.

Le funzioni principali sono:

- `semagraph_anchor_state64`: ancoraggio di sei osservazioni note;
- `semagraph_lookup_transition64`: lookup di una transizione;
- `semagraph_compress_chain64`: compressione di una catena ordinata;
- `semagraph_policy_context64`: costruzione del contesto per policy synthesis.

Il modello linguistico opera quindi come redattore di policy o spiegatore semantico, non come motore di verità osservativa.

7 Confronto con famiglie note

SemaGraph non è un motore fisico, non è un planner general-purpose, non è un motore BPMN e non è un rule engine generico. Ha parentele con automi finiti, statecharts, modelli di transizione, rule engines e policy-as-code, ma il suo scopo è più ristretto: ancorare stati osservati e comprimere catene di trasformazione in firme operative.

Famiglia	Somiglianza	Differenza
Automi finiti	Stati e transizioni finite	SemaGraph è orientato alla compressione di traiettorie osservate
Statecharts	Transizioni controllate e guardie	Non orchestra comportamento applicativo o UI
Rule engines	Predicati e regole	Non produce inferenza generale; valida transizioni e policy context
Policy-as-code	Regole operative formalizzabili	La policy è a valle della firma transizionale
Modelli stocastici	Traiettorie e percorsi	Non sceglie probabilisticamente il percorso; rappresenta deterministicamente i passaggi osservati
LLM toolchains	Output strutturato e validazione	LLM non è autorità su stati o transizioni

8 Ipotesi di validazione

La validazione non deve dimostrare che SemaGraph “descrive il mondo” in senso generale. Deve testare ipotesi operative:

1. riduce il tempo di classificazione di traiettorie osservate;
2. riduce chiamate LLM e token necessari per produrre policy coerenti;
3. aumenta replayability e auditabilità;
4. conserva informazione decisionale sufficiente rispetto alla traiettoria grezza;
5. evita falsi equivalenti prodotti da compressione eccessiva.

Una prima validazione può usare traiettorie sintetiche e confrontare tre modalità: analisi libera, analisi LLM-only e analisi SemaGraph con policy synthesis a valle.

9 Limiti

I principali limiti sono:

- la qualità del sistema dipende dalle regole di ancoraggio;
- la compressione può perdere informazioni rilevanti;
- due catene con uguale mutazione netta possono avere significato operativo diverso;
- il modello non sostituisce solutori numerici o modelli scientifici;
- la sintesi LLM delle policy richiede validazione umana o rule-based.

Il limite più critico è la perdita informativa. Per questo la firma compressa deve restare sempre collegata alla catena osservata e ai dati grezzi.

10 Conclusione

SemaGraph v0.6 formalizza un kernel deterministico per rappresentare catene di cambiamento in uno spazio finito di 64 stati booleani. La sua funzione non è predire, inferire o simulare direttamente la realtà, ma comprimere transizioni osservate in firme operative riproducibili. Questa struttura può ridurre inferenza ricorrente, migliorare auditabilità e abilitare policy synthesis controllata. La direzione Rust/WASM rende il nucleo compatibile con una futura esecuzione machine-near, mentre TypeScript resta il livello di specifica e integrazione.

Riferimenti bibliografici

- [1] C. E. Shannon, “A Mathematical Theory of Communication”, *Bell System Technical Journal*, 1948.
- [2] R. W. Hamming, “Error Detecting and Error Correcting Codes”, *Bell System Technical Journal*, 1950.
- [3] J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Pearson, 2006.
- [4] D. Harel, “Statecharts: A Visual Formalism for Complex Systems”, *Science of Computer Programming*, 1987.

- [5] R. Alur, D. L. Dill, “A Theory of Timed Automata”, *Theoretical Computer Science*, 1994.
- [6] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson, 2021.